

Hierarquia de Chomsky e Reconhecedores Sintáticos

MODELAGEM COMPUTACIONAL

Prof. Dr. José Jairo.

**Felipe Henrique Ribeiro
Luiz André Hoffmann Leineker**

**CURITIBA
2026**

SUMÁRIO

1. Introdução e contexto aplicado.....	3
2. Linguagem Regular.....	5
3. Linguagem Livre de Contexto	6
4. Linguagem Recursiva.....	8
5. Modelos formais e transições.....	9
6. Implementação.....	11
7. Testes e resultados	11
8. Comparação entre os três níveis da hierarquia.....	12
9. Conclusão e limitações.....	13

1. Introdução

A teoria da computação encontra na Hierarquia de Chomsky uma classificação fundamental para organizar as linguagens formais de acordo com a sua complexidade estrutural e a capacidade de memória exigida para seu processamento. Na prática do desenvolvimento de software, toda vez que um sistema decide se uma entrada é válida, ele atua como um reconhecedor de uma linguagem formal. Validadores de formulários, interpretadores de protocolos de rede e parsers de compiladores utilizam esses conceitos teóricos continuamente.

O objetivo deste projeto é construir e analisar um sistema aplicado de validação formal contendo três reconhecedores computacionais distintos, implementados em Python, sem o auxílio de bibliotecas prontas de parsing. Adotando o Tema 1, o sistema concentra-se na validação de dados cadastrais, expressões simbólicas e integridade de cadeias

2. Linguagem Regular

A primeira classe de linguagem abordada pertence ao Nível 3 da Hierarquia de Chomsky: a Linguagem Regular (LR).

- **Descrição em português:** Trata-se da validação da máscara e do formato estrito do Cadastro de Pessoas Físicas (CPF). O escopo restringe-se exclusivamente à validação da estrutura textual de caracteres e separadores posicionais, sem computar as operações aritméticas de validação dos dígitos verificadores finais.
- **Definição formal matemática:** A linguagem é definida pelo conjunto de cadeias sobre o alfabeto que seguem a regra de formação posicional fixa:

$$L_{CPF} = \{d_1d_2d_3.d_4d_5d_6.d_7d_8d_9 - d_{10}d_{11} \mid d_i \in \{0, 1, \dots, 9\}\}$$

- **Alfabeto utilizado:** (E): $E = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ., -\}$

3. Linguagem Livre de Contexto

Avançando para o Nível 2 da hierarquia, o sistema implementa uma solução capaz de lidar com estruturas aninhadas e dependência de escopo, características intrínsecas às Linguagens Livres de Contexto (LLC).

Descrição em português: Validação do balanceamento e simetria de parênteses, colchetes ou chaves em uma expressão simbólica. O

reconhecedor assegura que cada símbolo de abertura possua um caractere correspondente de fechamento respectivo e que nenhum fechamento ocorra antes da abertura equivalente.

Definição formal matemática: Definida por uma Gramática Livre de Contexto $G=(V,E, R,S)$, onde as regras de produção estruturam o aninhamento perfeito e correto dos caracteres.

4. Linguagem Recursiva

No Nível 0/1 da Hierarquia de Chomsky, encontram-se problemas que demandam capacidade de armazenamento irrestrito e movimentação livre de leitura/escrita para correlacionar posições distantes de memória, escopo coberto pelas Linguagens Recursivas (R).

- Descrição em português: Validação da cópia exata de uma cadeia binária com a presença de um caractere central delimitador, gerando o formato simétrico e sequencial de igualdade de blocos.
- Definição formal matemática: A linguagem é expressa formalmente por:

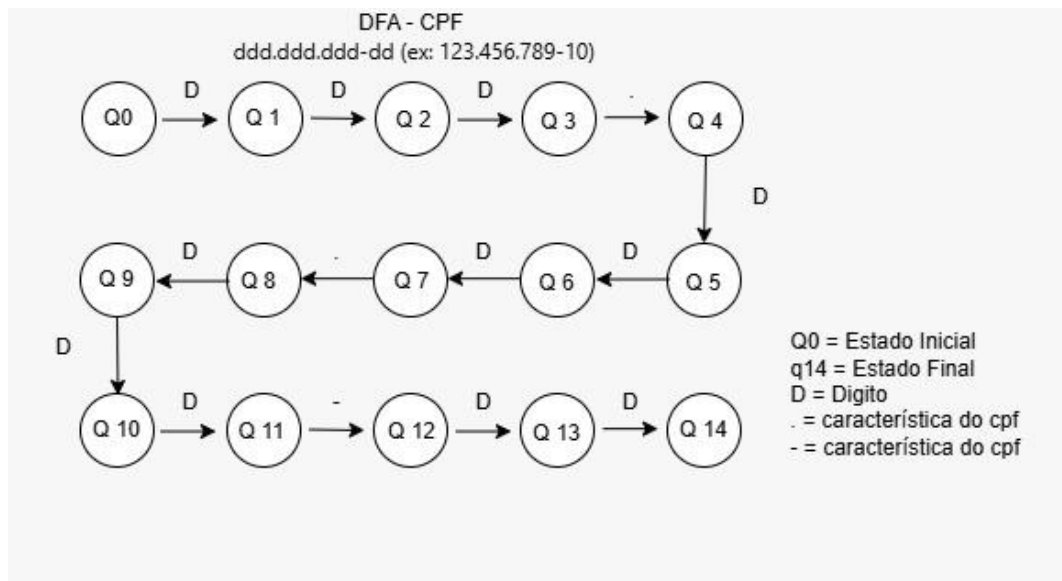
$$L = \{w\#w \mid w \in \{0,1\}^*\}$$

5. Modelos formais e transições

Cada reconhecedor foi mapeado para o seu respectivo modelo abstrato ideal de computação, variando os estados e o uso de recursos de memória:

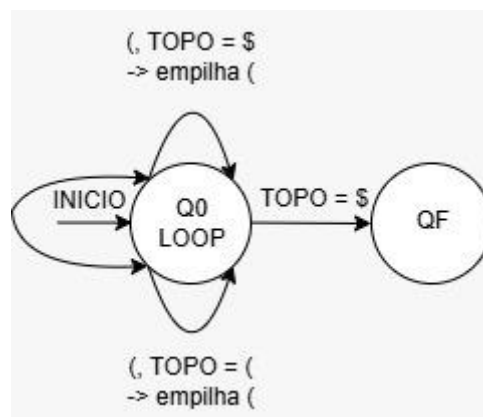
Autômato Finito Determinístico (DFA) – Validador de CPF

O DFA opera de forma estritamente linear. Possui um conjunto de estados $Q=\{q_0,q_1,...,q_{14},q_{error}\}$. As transições seguem uma sequência fixa na qual os estados 0, 1, 2, 4, 5, 6, 8, 9, 10, 12, 13 avançam apenas com a leitura de dígitos numéricos (`char.isdigit()`). Os estados 3 e 7 realizam a transição se o caractere lido for estritamente um ponto (`.`), e o estado 11 avança unicamente com o hífen (`-`). Qualquer desvio desse caminho direciona o autômato para `q_error`. O estado de aceitação final é atingido apenas ao consolidar o estado 14 ao término da leitura.



- Autômato com Pilha Determinístico (PDA) – Balanceamento

O PDA utiliza um controle de estados associado a uma pilha virtual. Ao iniciar no estado q_0 , a leitura do caractere verifica se a pilha contém elementos; se sim, realiza a operação de desempilhamento (pop). Se a pilha estiver vazia no momento do desempilhamento, o autômato entra em estado de rejeição imediata. O critério de aceitação é a leitura completa da string associada ao retorno da pilha ao seu estado inicial totalmente vazio.



- Máquina de Turing Determinística (MT) – Cópia de Cadeia $\$w\#w\$$

A máquina simula uma fita bidirecional contendo células delimitadas por símbolos de espaço em branco (B) nas extremidades. No estado inicial q_0 , o cabeçote lê o primeiro caractere à esquerda do separador #. Se ler 0, substitui por X (marcando o caractere como lido) e altera seu estado para q_{look_0} , movendo o cabeçote para a direita passando por cima de outros dígitos e pelo caractere #. Ao

cruzar o #, o cabeçote busca o primeiro caractere não marcado; se for 0, este é substituído por um símbolo neutro de validação (V) e o autômato altera seu estado para q_return, movendo o cabeçote de volta para a esquerda até encontrar a última marcação. O processo se repete de forma análoga para o caractere 1 (substituindo por Y). Se todas as marcações forem validadas de forma idêntica e sem sobras em ambos os lados, a máquina atinge o estado final q_accept.

6. Implementação

A arquitetura implementada no repositório de software ValidadorFinal é pautada na separação modular orientada a objetos. Cada reconhecedor foi desenvolvido de maneira puramente algorítmica, sem herança de pacotes externos.

A função de transição de cada modelo foi mapeada explicitamente como dados e regras de controle dentro de laços de repetição e condicionais (if/else) que avaliam símbolo por símbolo da entrada. Obedecendo às restrições do projeto, o contador de passos foi implementado incrementando uma variável de controle de forma restrita às transições efetivas do modelo de automação, não computando iterações gerais de loops secundários do código Python.

7. Testes e resultados

Para homologação do comportamento lógico do sistema, o software executa uma bateria de testes verificando cadeias válidas e inválidas para cada um dos modelos. Os resultados experimentais obtidos e o comportamento do contador de passos estão sumarizados na tabela de conformidade abaixo:

Módulo do Validador	Cadeia de Entrada (Input)	Status Esperado	Resultado Obtido
DFA (Finito)	"123.456.789-10"	ACEITA	ACEITA (Sucesso)
DFA (Finito)	"111222333-44"	REJEITADA	REJEITADA (Sucesso)
DFA (Finito)	"abc.def.ghi-jk"	REJEITADA	REJEITADA (Sucesso)
PDA (Pilha)	"()"	ACEITA	ACEITA (Sucesso)
PDA (Pilha)	"(())"	REJEITADA	REJEITADA (Sucesso)
PDA (Pilha)	"())"	REJEITADA	REJEITADA (Sucesso)
MT (Turing)	"101#101"	ACEITA	ACEITA (Sucesso)
MT (Turing)	"01#10"	REJEITADA	REJEITADA (Sucesso)
MT (Turing)	"110#"	REJEITADA	REJEITADA (Sucesso)

8. Comparação entre os três níveis de hierarquia

A execução prática dos três módulos deixa evidente a gradação de capacidade analítica descrita pela teoria da computação:

- O DFA (Linguagem Regular) possui tempo de execução estritamente linear e previsível, porém é incapaz de guardar históricos estruturais dinâmicos. Ele funciona perfeitamente para checar se símbolos pré-definidos ocupam posições exatas, mas falharia caso a string exigisse memória de contagem ou tamanho indeterminado.
- O PDA (Linguagem Livre de Contexto) supera a limitação do DFA ao introduzir uma memória auxiliar em forma de pilha. Isso permite o reconhecimento de aninhamentos de profundidade indefinida, contanto que o critério de validação siga uma ordem inversa de acesso. O PDA falha, entretanto, se o problema exigir a comparação de elementos em ordem direta ou o cruzamento de dados de múltiplos escopos isolados.
- A Máquina de Turing (Linguagem Recursiva) soluciona as restrições anteriores ao quebrar a linearidade de acesso. Ao permitir que o cabeçote navegue livremente pela fita reescrevendo os dados, o modelo consegue indexar, parear e validar cadeias de alta complexidade estrutural, como a igualdade de blocos isolados $W=W$.

9. Conclusão e limitações

A execução do projeto comprova de forma empírica e experimental que a estrutura de dados interna utilizada por um validador dita diretamente a classe de problemas que ele é capaz de solucionar. A modelagem pura dos autômatos em Python permitiu fixar os conceitos matemáticos estudados sem a abstração de engines comerciais de Expressões Regulares.

Como limitações do sistema desenvolvido, cita-se o aumento do custo computacional e de passos da Máquina de Turing, cujo comportamento de varredura de fita em "vai e vem" gera complexidade de tempo polinomial superior em comparação ao processamento puramente linear do DFA e do PDA. Adicionalmente, o sistema opera sob uma abordagem estritamente determinística, necessitando de reestruturação completa de tabelas de transição caso fosse exposto a gramáticas ambíguas ou linguagens não-determinísticas.